Stockly

PLATAFORMA DE INVENTARIO · ES-CO / EN

Un inventario que se vende solo

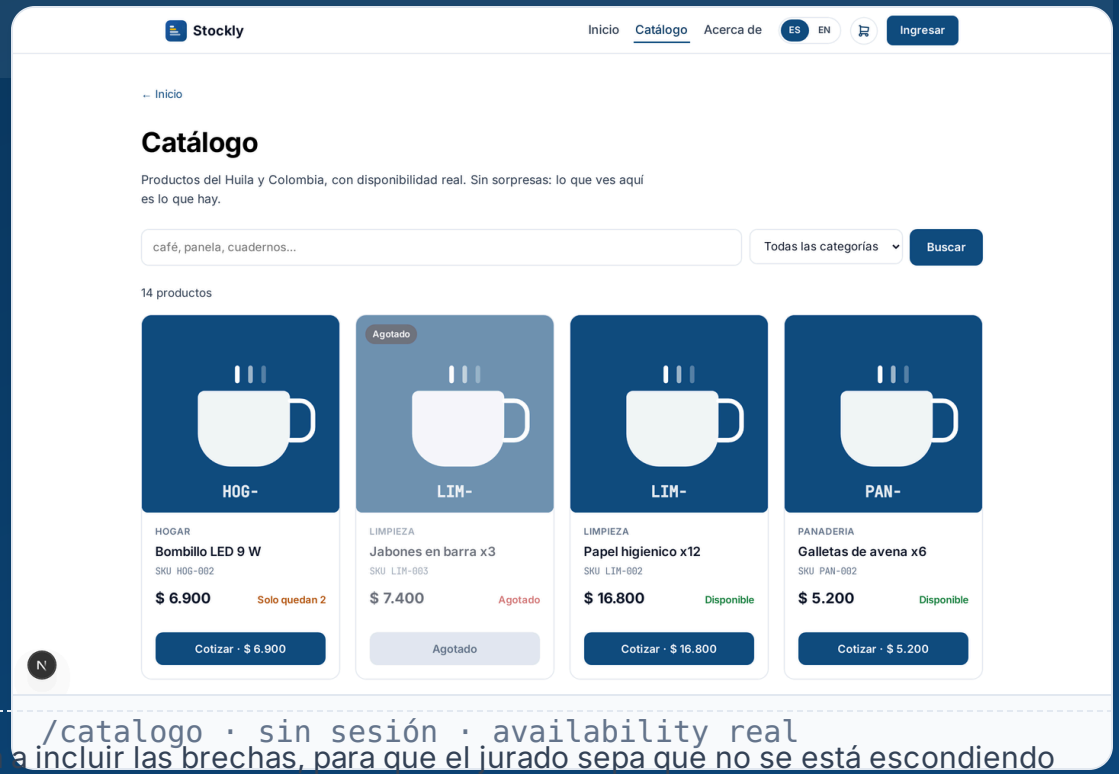
La tienda es pública. El panel es privado. Detrás hay un contrato de 75 requisitos versionado, verificado contra la aplicación en ejecución —y sus 15 brechas, declaradas una por una.

Juan Nicolas Urrutia Salcedo
Luisa Maria Jose Coronel Vargas
Ana Sofía Montaña Cedeño
Universidad Surcolombiana de Neiva · Materia Inteligencia Artificial · 2026

LA RESPUESTA PRIMERO

Un comprador sin cuenta ve el stock real. Un operador con cuenta lo cambia. Nadie ve lo que no existe.

- **Catálogo público** — 6 rutas sin sesión.
- **Panel de administración** — 6 rutas, con control de rol en el servidor.
- **Disponibilidad honesta** — «Solo quedan 2», «Agotado». Nunca urgencia fabricada.



/catalogo · sin sesión · availability real

Marcar el ritmo de los tres bloques aquí. Decir explícitamente que el bloque 3 va a incluir las brechas, para que el jurado sepa que no se está escondiendo nada y escuche el resto con esa expectativa.

EL PROBLEMA

Una hoja de cálculo rompe en tres sitios

01 · Vender lo que no hay

La hoja no se actualiza al vender. Llega el comprador y se le **niega el producto** que vio publicado.

02 · Nadie mira la página

Si el precio y las existencias **no se publican**, la venta se pierde antes de empezar. No hay a dónde ir.

03 · Nadie sabe qué pasó

Sin registro de cambios, cuando algo sale mal **no hay forma de saber** qué pasó ni cuándo.

Stockly ataca los tres con la misma tabla: el catálogo que ve el comprador y el panel que edita el operador leen y escriben el mismo dato.

No leer las tres tarjetas: nombrarlas y quedarse en la tercera. La tercera es la que conecta con todo el bloque 2 (auditoría) y con el bloque 3 (trazabilidad). Si hay tiempo, preguntar al jurado cuál les parece el más grave.

LA CORRECCIÓN DE RUMBO

Primero construimos un panel. Después: faltaba media tienda

EL DIAGNÓSTICO

Todo lo construido estaba del lado del operador. **El dinero no tenía dónde ocurrir.** Un panel perfecto donde nadie compra.

- El catálogo pasó a ser **público, sin cuenta**.
- La disponibilidad se hizo **visible en la ficha**, no escondida en la base.
- Y de esa decisión salió una regla: **la honestidad del estado es un requisito**, no una cortesía visual.

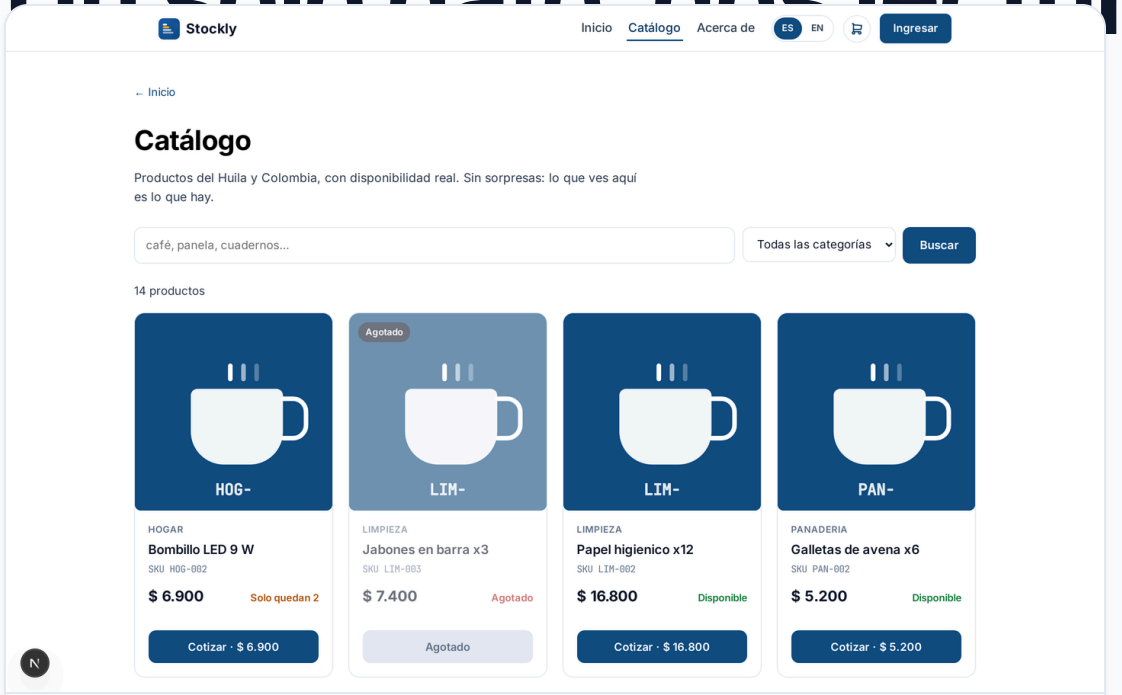


`/admin · 6 indicadores · declara el recorte de 100`

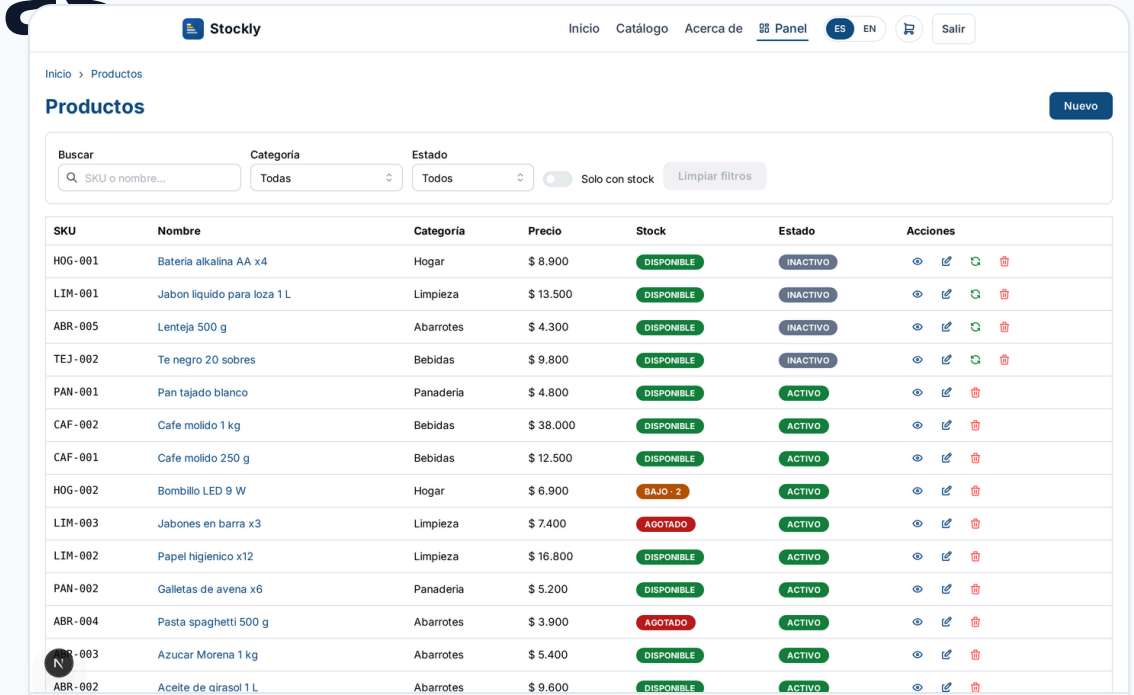
Es el momento más honesto de la exposición: admitir que el producto estaba a medias. Contar que la corrección llegó por un diagnóstico, no por requisito, y que se resolvió con una especificación derivada en vez de reescribir el contrato original — porque reescribir un contrato vivo rompe su trazabilidad. Señalar el pie de la captura: los indicadores declaran que se calculan sobre los primeros 100 registros.

LAS DOS SUPERFICIES

Un solo dato, dos lecturas



Catálogo · lo que ve el comprador



Panel · lo que edita el operador

La columna `stock` es la misma en las dos. «AGOTADO» y «BAJO · 2» no son adornos: son la regla de negocio **RN-11** dizendo la verdad.

Señalar el botón de reactivar (flecha verde) en la columna de acciones: existe porque la baja es lógica, no borrado físico. Un producto dado de baja sigue en la base y se puede volver a activar. Es RN-10, y es lo que hace reversible un error de teclado.

LO QUE ACABA DE VERSE

El producto existe, se corrigió a mitad de camino, y su regla central es no mentir sobre el estado.

Lo que sigue no es cómo se ve, sino **cómo se construyó y cómo se comprobó**. Empieza Luisa.

Tránsito limpio. No añadir contenido nuevo: solo pasar la palabra. El bloque 2 es de Luisa, el bloque 3 es de Ana.

LA RESPUESTA PRIMERO

Tres capas, un contrato de requisitos versionado y ninguna regla de negocio en el cliente.

01 · Stack real

Next.js 15 · React 19 · TypeScript **estricto**
Spring Boot 3.4 · Java 17 · JPA
PostgreSQL 16 · Redis 7 · Liquibase

02 · Contrato

75 requisitos con identificador estable
15 tareas · 5 especificaciones
6 migraciones versionadas

03 · Autoridad

Cada regla se valida **en el servidor**.
Ocultar el botón no es seguridad.

No es un listado de tecnologías: cada tarjeta responde una decisión. Si preguntan por la elección, decir que cada una está justificada en la documentación con la alternativa descartada y el motivo (docs/02-arquitectura.md §7, 12 decisiones).

Cuatro decisiones que no son obvias

- **El SKU es inmutable.** El comercio puede tenerlo impreso en una etiqueta; cambiarlo rompe una referencia que ya existe fuera del sistema.
- **Nada se borra de verdad.** Dar de baja marca `active=false`. Una eliminación accidental es reversible.
- **El token de acceso vive solo en memoria.** El almacenamiento local es legible por cualquier script inyectado: un XSS se vuelve robo de sesión.
- **Los errores son datos.** RFC 7807 con un `type` estable. El cliente reacciona al código, nunca al texto.

Cada una tiene una alternativa que se consideró y se descartó por escrito. Sin el motivo escrito, dentro de seis meses nadie recuerda por qué se hizo así.

Estas cuatro son las que el jurado va a preguntar. Para cada una: la alternativa descartada y el coste quePagamos. La del SKU tiene un coste: corregir un SKU mal capturado requiere una operación de soporte específica. Reconocer el coste es lo que hace creíble la decisión.

SESIÓN Y SEGURIDAD

Un fallo que costó una sesión legítima

1
El acceso expira a los 15 min.
Varias peticiones fallan a la vez.

2
Cada una dispara su propia renovación con el mismo token.

3
El token es de un solo uso: la segunda se lee como **reutilización maliciosa**.

4
La familia de tokens queda revocada. **Sesión legítima muerta.**

La corrección no fue un parche: fue **coalescer las renovaciones** en una sola operación en vuelo. Es un bug que solo aparece cuando el sistema se usa de verdad —y por eso la demostración en vivo importa.

`acceso 15 min · en memoria` `refresco 7 días · cookie HttpOnly` `rotación de un solo uso` `reutilizar → 401`

Esta es la lámina que demuestra criterio de ingeniería: un fallo de concurrencia con síntoma de seguridad. Explicar que la versión «ingeniosa» sin coalescencia habría sido detectada como ataque y haber matado al usuario legítimo. Si sobra tiempo: mencionar que la revocación se guarda en Redis con TTL igual al del token, para que no crezca sin límite.

LO QUE SE CONSTRUYÓ

El sistema, medido

75
requisitos catalogados
con identificador estable

6
migraciones Liquibase
aplicadas y versionadas

10
operaciones de API
5 catálogo + 5 sesión

2
idiomas con paridad
completa
es-CO / en

11
rutas de la aplicación
6 públicas · 5 de panel

8
manejadores de error
todos en RFC 7807

12
decisiones arquitectónicas
con alternativa descartada

6
indicadores en el panel
que declaran su recorte

Señalar que 11 rutas = 5 públicas + 6 de panel (la página de alta y la de edición). No leer las ocho cifras: dejarlas y highlight las dos que se explican en voz alta. Advertir al jurado que la honestidad de estas cifras es el tema del bloque 3.

LO QUE ACABA DE VERSE

El sistema está construido con reglas explícitas, y cada regla tiene un motivo escrito.

Ahora la parte que más cuesta decir: **qué está verificado y qué no**. Sigue Ana.

Tránsito. El bloque 3 es el incomodo y hay que anunciarlo con naturalidad, no como disculpa: es la lámina que da credibilidad a todo lo anterior.

LA RESPUESTA PRIMERO

38 pruebas end-to-end en verde y cero pruebas de backend.

POR QUÉ LO DIGO PRIMERO

Es lo primero que va a preguntar el jurado, así que conviene que la respuesta la tengan antes de que la formulen.

El sistema **funciona**: se verificó endpoint por endpoint contra la aplicación levantada. Lo que no existe es la **red que lo vuelva a demostrar solo**.

Decir la cifra sin rodeos y sin disculpa. La distinction clave: «funciona» y «está verificado» no son lo mismo, y la diferencia es exactamente el trabajo que falta. Si el jurado pregunta cuántos requisitos están cumplidos: 75 catalogados, 60 cumplidos, 4 parciales, 8 pendientes, 3 configurados pero nunca ejecutados.

DÓNDE SÍ HAY EVIDENCIA

Lo que se puede volver a ejecutar hoy

38 · E2E en navegador real

Escritorio y móvil. Se autentican por la interfaz, como una persona.

0 · violaciones graves de accesibilidad

axe-core corre **dentro** de cada prueba, no como revisión aparte.

11 · rutas con estados propios

Carga, error y no encontrado. Nunca una pantalla en blanco.

Y hay dos cosas más que **sí son comprobables sin pruebas**: la sesión (rotación de un solo uso, verificada a mano devolviendo `401`) y la autorización (usuario común → `403`, anónimo → `401`).

Esta lámina equilibra la anterior. Aclarar un matiz técnico que impresiona: los tests no usan storageState porque el token vive en memoria, así que cada prueba se autentica por la UI. Es una limitación del diseño de sesión que termina siendo una ventaja: la prueba recorre el flujo real de login.

LA BRECHA CENTRAL

Configurado no es ejecutado

80%

cobertura mínima de líneas
configurada, nunca ejecutada

60%

puntaje de mutación mínimo
configurado, nunca ejecutado

0

pruebas de backend
ni una

0

ejecuciones en integración continua
no existe

Y ESTO ES LO QUE MÁS PESA

Sin integración continua, las otras tres compuertas son **configuración que nadie dispara**. Un umbral que nunca corre no es un umbral: es un mensaje de intenciones.

Aquí está el argumento central del bloque. La frase para el jurado: un umbral de cobertura que nunca se ejecuta no protege nada. Construir la integración continua es un archivo de configuración y dos órdenes, y es lo que desbloquea todo lo demás.

EL PLAN, EN ORDEN

Qué haríamos primero

- **1 · Integración continua.** Sin ella nada más se ejecuta solo.
- **2 · Pruebas de backend** con PostgreSQL real en contenedor.
- **3 · Matriz de trazabilidad** requisito ↔ prueba, en ambos sentidos.
- **4 · Análisis de dependencias.** La herramienta ya está configurada: es una orden.
- **5 · Medición de usabilidad** con una persona real. Se puede hacer en paralelo.

El orden no es por dificultad, es por **dependencia**: cada paso hace posible el siguiente. Hay 15 brechas registradas en total; estas son las cinco con mayor relación entre esfuerzo y valor.

El paso 5 merece una frase: la medición de usabilidad (SUS ≥ 68, tasa de éxito ≥ 90 %) es la única que no depende de una herramienta, depende de una persona. El protocolo ya está escrito; falta una sesión de 40 minutos con un participante que nunca haya visto el sistema.

CIERRE

Lo que queremos que se lleven

- Un sistema **construido con reglas explícitas**, no por acumulación.
- Una documentación que **se contradice a sí misma** y por eso es creíble.
- Y un plan que empieza por lo que hace posible todo lo demás.

DOCUMENTACIÓN ABIERTA

docs/README.md — por dónde empezar
01-requisitos.md — 75 requisitos con estado
04-estado-y-brechas.md — las 15 brechas
assets/presentacion/ — estas capturas

```
Aplicación corriendo en http://localhost:3000
Licencia MIT · Código abierto
```

Cerrar volviendo al mensaje del principio: el bloque 3 existe para que el bloque 1 y 2 valgan. Si hay tiempo para preguntas, las tres más probables son: (1) por qué no hay pruebas unitarias de backend —respuesta: tiempo, y el orden del plan las pone primeras; (2) por qué el SKU es inmutable; (3) qué pasa con la concurrency de sesiones.